Miskolci Egyetem

Kiértékelés: N X N-es mátrix szorzása OpenCL segítségével

Készítette:
Turi Zsófia Eszter
Programtervező informatikus

Miskolc, 2026

Tartalomjegyzék

1. Bevezetés a mátrixszorzás történetébe 2
2. n = 256 mátrix 3
   2.1. Mit látunk? 3
   2.2. Ez azt mutatja, hogy: 3
3. n = 512 mátrix 4
   3.1. Mit látunk? 4
   3.2. Ez azt mutatja, hogy: 4
4. n = 1024 mátrix 5
   4.1. Mit látunk? 5
   4.2. Ez azt mutatja, hogy: 5
5. n = 2048 mátrix 6
   5.1. Mit látunk? 6
   5.2. Ez azt mutatja, hogy: 6
6. Futtatási eredmények diagramjai 7
   6.1. 1.ábra magyarázata 7
   6.2. 2.ábra magyarázata 8
   6.3. 3.ábra magyarázata 9
   6.4. 4.ábra magyarázata 10
   6.5. 5.ábra magyarázata 11
   6.6. 6.ábra magyarázata 12
   6.7. 7.ábra magyarázata 13
7. Összefoglalás 14

Irodalomjegyzék **15**

1. Bevezetés a mátrixszorzás történetébe

A mátrixszorzás a lineáris algebra egyik legalapvetőbb művelete, amelynek jelentősége messze túlmutat a tiszta matematikán: a modern számítástechnika, a gépi tanulás, a számítógépes grafika, a fizikai szimulációk és a nagyméretű adatfeldolgozás mind erre a műveletre épül. A klasszikus mátrixszorzási algoritmus már a 19. század végén kialakult, és hosszú időn át megkérdőjelezhetetlen alapnak számított. Ennek időbeli bonyolultsága ON^3, ami nagy dimenziók esetén gyorsan a számítási kapacitás határaiba ütközik [golub].

A 20. század közepére a számítógépek elterjedése új lendületet adott a hatékonyabb módszerek keresésének. A valódi áttörést Volker Strassen 1969-es eredménye hozta, aki bebizonyította, hogy a mátrixszorzás elvégezhető a klasszikus módszernél kevesebb művelettel. Strassen algoritmusa O$N^{2.81}$ időkomplexitást ért el, ami elméleti szempontból forradalmi volt, és megnyitotta az utat a még gyorsabb, bár gyakran csak elméleti jelentőségű algoritmusok előtt [strassen].

A modern számítástechnika fejlődése azonban új irányt jelölt ki: a hangsúly az algoritmikus optimalizálásról a párhuzamosításra helyeződött át. A CPU-k hagyományosan egymásba ágyazott ciklusokkal, egy vagy néhány szálon számolják ki a mátrixszorzatot, míg a GPU-k több ezer párhuzamos szálat indítanak, amelyek mindegyike egyetlen mátrixelem kiszámításáért felel. Ez a szemléletváltás tette lehetővé, hogy a mátrixszorzás – és vele együtt a rá épülő alkalmazások – nagyságrendekkel gyorsabbá váljanak, különösen olyan területeken, mint a mélytanulás vagy a valós idejű grafikai feldolgozás [cuda_article][cuda_guide].

A mátrixszorzás története így nem csupán matematikai fejlődéstörténet, hanem a számítástechnika evolúciójának lenyomata is: jól mutatja, hogyan alakítja a hardver és a szoftver kölcsönhatása a modern számítási módszereket.

Műveletszám:

Szorzások: N^3
Összeadások: $N^3 - N^2$
Összesen: $2N^3 - N^2$ művelet

2. n = 256 mátrix

A következő fejezet az n = 256 méretű mátrixszorzás CPU–GPU futási eredményeit mutatja be. Az elemzés az Listing 1 mérési adataira épül, amelyek tartalmazzák a CPU futási idejét, a GPU által indított szálak számát, a kernel futási idejét, valamint a teljes GPU futási időt (adatmásolással együtt).

2.1. Mit látunk?

- A CPU futásideje: 0.01 s
- A GPU kernel futásideje: 0.000417 s
- Teljes GPU idő (másolással): 0.000669 s
- 1 szál futási ideje: 0.000000006360 s
- Gyorsítás:
  - CPU / kernel: 23.99×
  - CPU / teljes GPU: 14.96×

2.2. Ez azt mutatja, hogy:

- A GPU már ebben a méretben is gyorsabb, mint a CPU.
- A másolási költség (Host - Device) észrevehető, de nem domináns.
- Számítások:
  - Műveletek száma: $2 \cdot 256^3 - 256^2$ = 33.488.896 művelet
  - GPU: 256^2 szál = 33.488.896 művelet / 65.536 szál $\rightarrow$ 511 művelet / 1 szál
  - CPU: 1szál = 33.488.896 művelet / 1 szál

Listing 1. Futási eredmények

```
CPU time: 0.010000 s
GPU threads: 256 x 256 = 65536
GPU kernel time: 0.000417 s
Total GPU time: 0.000669 s
Speedup (CPU / kernel): 23.99
Speedup (CPU / total GPU): 14.96
Time per thread: 0.000000006360 s
```

3. n = 512 mátrix

A következő fejezet az n = 512 méretű mátrixszorzás CPU–GPU futási eredményeit mutatja be. Az elemzés az Listing 2 mérési adataira épül, amelyek tartalmazzák a CPU futási idejét, a GPU által indított szálak számát, a kernel futási idejét, valamint a teljes GPU futási időt (adatmásolással együtt).

3.1. Mit látunk?

- A CPU futásideje: 0.127000 s
- A GPU kernel futásideje: 0.002741 s
- Teljes GPU idő (másolással): 0.003932 s
- 1 szál futási ideje: 0.000000010456
- Gyorsítás:
  - CPU / kernel: 46.34×
  - CPU / teljes GPU: 32.30×

3.2. Ez azt mutatja, hogy:

- A GPU ebben a méretben már jelentős gyorsulást biztosít a CPU-hoz képest.
- A másolási költség (Host - Device) jelen van, de nem dominálja a teljes futási időt.
- Számítások:
  - Műveletek száma: $2 \cdot 512^3 - 512^2$ = 268.173.312 művelet
  - GPU: 512^2 szál = 268.173.312 művelet / 262.144 szál $\rightarrow$ 1023 művelet / 1 szál
  - CPU: 1 szál = 268.173.312 művelet / 1 szál

Listing 2. Futási eredmények (n = 512)

```
CPU time: 0.127000 s
GPU threads: 512 x 512 = 262144
GPU kernel time: 0.002741 s
Total GPU time: 0.003932 s
Speedup (CPU / kernel): 46.34
Speedup (CPU / total GPU): 32.30
Time per thread: 0.000000010456 s
```

4. n = 1024 mátrix

A következő fejezet az n = 1024 méretű mátrixszorzás CPU–GPU futási eredményeit mutatja be. Az elemzés az Listing 3 mérési adataira épül, amelyek tartalmazzák a CPU futási idejét, a GPU által indított szálak számát, a kernel futási idejét, valamint a teljes GPU futási időt (adatmásolással együtt).

4.1. Mit látunk?

- A CPU futásideje: 3.321000 s
- A GPU kernel futásideje: 0.026434 s
- Teljes GPU idő (másolással): 0.033978 s
- 1 szál futási ideje: 0.000000025209 s
- Gyorsítás:
  - CPU / kernel: 125.63×
  - CPU / teljes GPU: 97.74×

4.2. Ez azt mutatja, hogy:

- A GPU ennél a méretnél már nagyságrendekkel gyorsabb, mint a CPU.
- A másolási költség (Host - Device) továbbra is jelen van, de egyre kisebb arányt képvisel a teljes futási időben.
- Számítások:
  - Műveletek száma: $2 \cdot 1024^3 - 1024^2$ = 2.145.386.496 művelet
  - GPU: 1024^2 szál = 2.145.386.496 művelet / 1.048.576 szál $\rightarrow$ 2047 művelet / 1 szál
  - CPU: 1 szál = 2.145.386.496 művelet / 1 szál

Listing 3. Futási eredmények (n = 1024)

```
CPU time: 3.321000 s
GPU threads: 1024 x 1024 = 1048576
GPU kernel time: 0.026434 s
Total GPU time: 0.033978 s
Speedup (CPU / kernel): 125.63
Speedup (CPU / total GPU): 97.74
Time per thread: 0.000000025209 s
```

5. n = 2048 mátrix

A következő fejezet az n = 2048 méretű mátrixszorzás CPU–GPU futási eredményeit mutatja be. Az elemzés az Listing 4 mérési adataira épül, amelyek tartalmazzák a CPU futási idejét, a GPU által indított szálak számát, a kernel futási idejét, valamint a teljes GPU futási időt (adatmásolással együtt).

5.1. Mit látunk?

- A CPU futásideje: 149.640000 s
- A GPU kernel futásideje: 0.442029 s
- Teljes GPU idő (másolással): 0.470762 s
- 1 szál futási ideje: 0.000000105388 s
- Gyorsítás:
  - CPU / kernel: 338.53×
  - CPU / teljes GPU: 317.87×

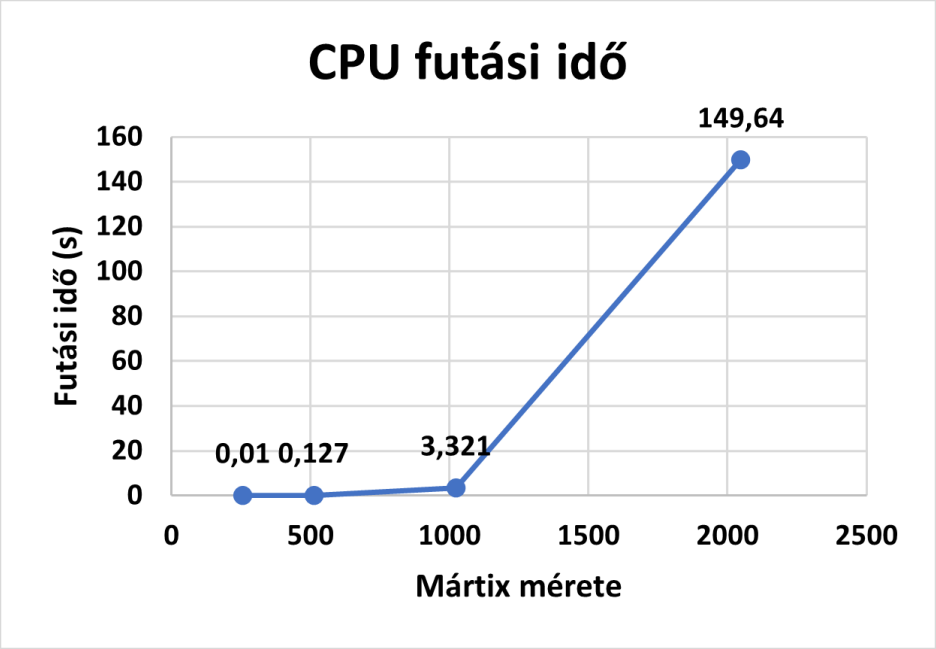
5.2. Ez azt mutatja, hogy:

- A GPU ennél a méretnél már drasztikus gyorsulást biztosít a CPU-hoz képest.
- A másolási költség (Host - Device) abszolút értékben nő, de aránya továbbra is alacsony.
- Számítások:
  - Műveletek száma: $2 \cdot 2048^3 - 2048^2$ = 17.175.560.192 művelet
  - GPU: 2048^2 szál = 17.175.560.192 művelet / 4.194.304 szál $\rightarrow$ 4095 művelet / 1 szál
  - CPU: 1 szál = 17.175.560.192 művelet / 1 szál

Listing 4. Futási eredmények (n = 2048)

```
CPU time: 149.640000 s
GPU threads: 2048 x 2048 = 4194304
GPU kernel time: 0.442029 s
Total GPU time: 0.470762 s
Speedup (CPU / kernel): 338.53
Speedup (CPU / total GPU): 317.87
Time per thread: 0.000000105388 s
```

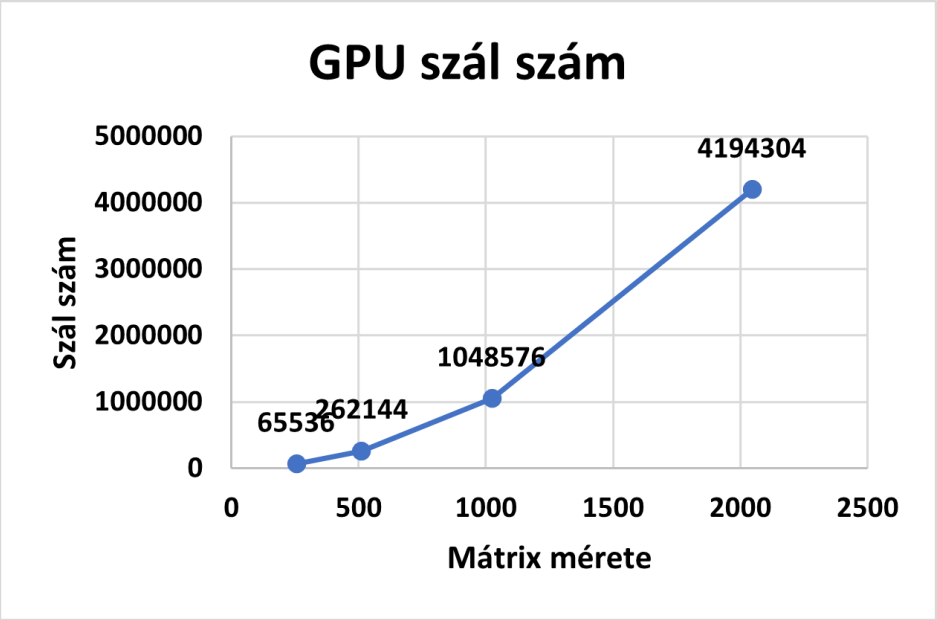
6. Futtatási eredmények diagramjai



1. ábra. CPU futási ideje

6.1. 1.ábra magyarázata

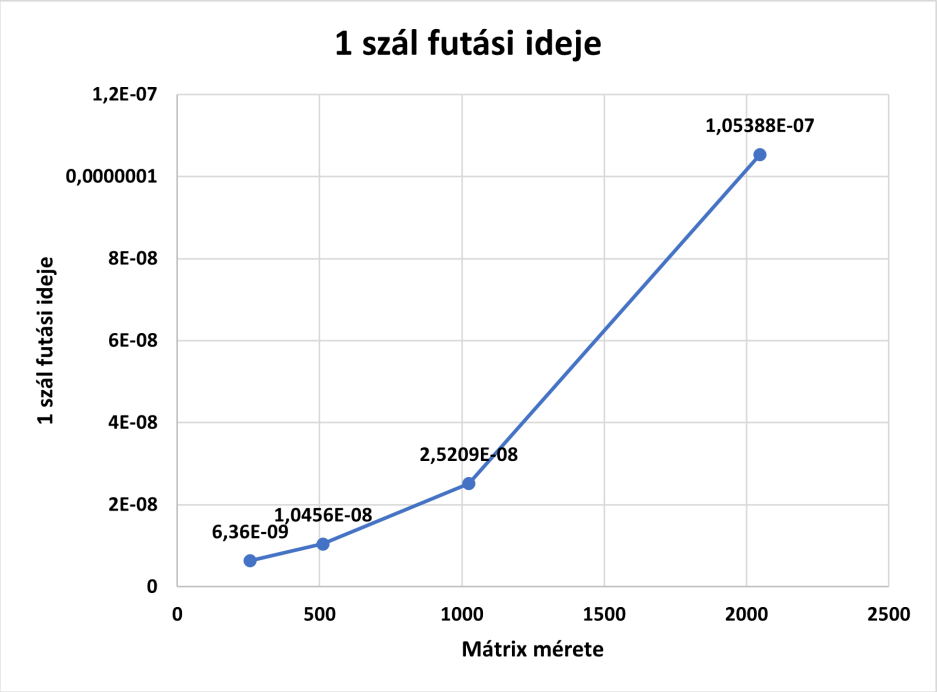
A grafikon azt mutatja, hogy a CPU mennyire gyorsan kezd el lelassulni, amikor egyre nagyobb mátrixokat kell összeszoroznia. Kisebb méreteknél (pl. 256×256) még szinte észrevehetetlen a terhelés, a futási idő csak néhány századmásodperc. Ahogy azonban nő a mátrix mérete, a CPU-nak drasztikusan több műveletet kell elvégeznie, mert a számítási igény nő. Emiatt 512-es méretnél már tizedmásodperceket látunk, 1024-nél több másodpercet, 2048-nál pedig a futási idő gyakorlatilag „elszáll”, és több mint két percig tart. A CPU egyszerűen nem tud lépést tartani ekkora adatmennyiséggel, mert a műveleteket egymás után, kevés szálon végzi. A grafikon így szépen megmutatja, hogy a CPU kis mátrixokra még alkalmas, de nagyobb méreteknél már nagyon lassúvá válik. Ezért használják a modern, nagy számításigényű feladatokban inkább a GPU-t, amely sokkal jobban kezeli a párhuzamos munkát.



2. ábra. Teljes GPU futási ideje

6.2. 2.ábra magyarázata

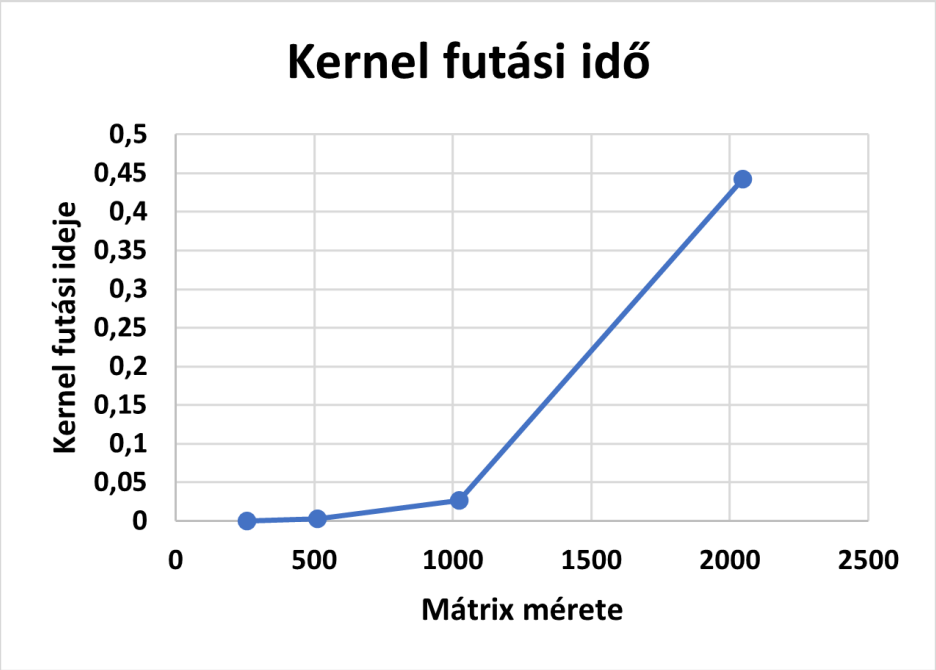
A grafikon azt mutatja, hogy a GPU hány párhuzamos szálat indít el különböző mátrixméreteknél. Mivel a GPU úgy működik, hogy minden kimeneti mátrixelemhez egy külön szálat indít, a szálak száma a mátrix méretével együtt négyzetesen nő. Ezért látjuk azt, hogy 256-os méretnél még „csak” 65 ezer szál fut, 512-nél már több mint 260 ezer, 1024-nél több mint egymillió, 2048-nál pedig már több mint 4 millió. A grafikon lényege az, hogy a GPU nagyon könnyen tud rengeteg szálat kezelni egyszerre — ez a működésének alapja. Míg a CPU néhány szálon dolgozik, addig a GPU akár milliós nagyságrendű párhuzamos feladatot is gond nélkül elindít. Ezért képes sokkal gyorsabban elvégezni a mátrixszorzást: a munka nem egy szálon fut végig, hanem több millió apró részfeladatra oszlik, amelyek egyszerre futnak. Összességében a grafikon azt üzeni, hogy a GPU teljesítménye a mátrixméret növekedésével egyre jobban érvényesül, mert minél nagyobb a mátrix, annál több szálat tud párhuzamosan futtatni — és ez az, amiben igazán erős.



3. ábra. 1 szál futási ideje

6.3. 3.ábra magyarázata

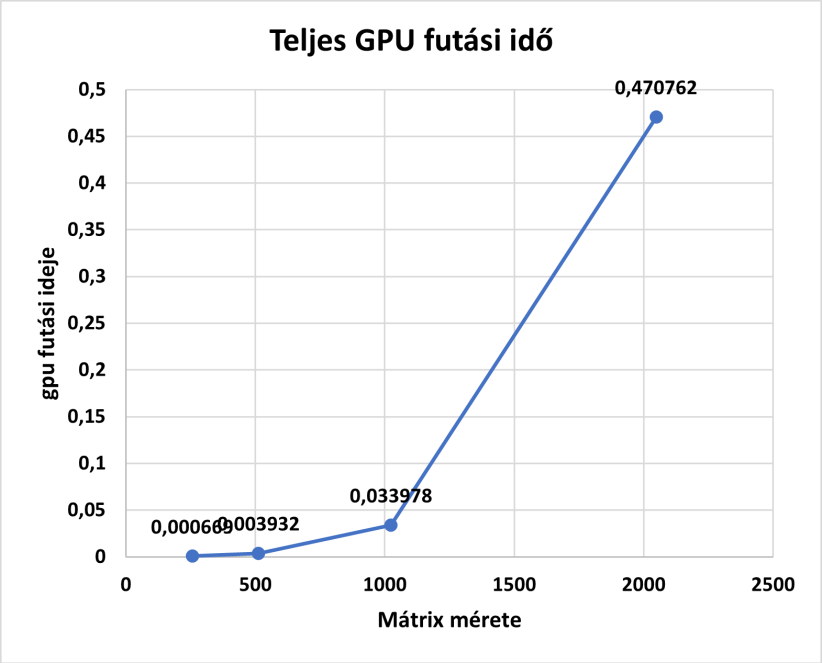
Ez a grafikon azt mutatja meg, hogy egyetlen GPU-szál mennyi idő alatt végzi el a rá jutó számítási munkát különböző mátrixméreteknél. A görbe enyhén emelkedik, ami azt jelzi, hogy ahogy nő a mátrix mérete, egy szálnak is egyre több műveletet kell elvégeznie. A növekedés azonban nagyon lassú. Ez azért van, mert a GPU úgy osztja szét a feladatot, hogy minden szál csak egy apró részfeladatot kap, így egy szálra nagyon kevés munka jut. A grafikon tehát azt üzeni, hogy a GPU szálai rendkívül gyorsak és hatékonyak: még nagy mátrixok esetén is szinte elhanyagolható idő alatt végzik el a saját részfeladatukat. Ez jól mutatja, miért működik olyan jól a GPU a mátrixszorzásnál — nem az egy szál ereje a lényeg, hanem az, hogy több tízezer vagy akár több millió ilyen gyors szál dolgozik egyszerre.



4. ábra. Kernel futási ideje

6.4. 4.ábra magyarázata

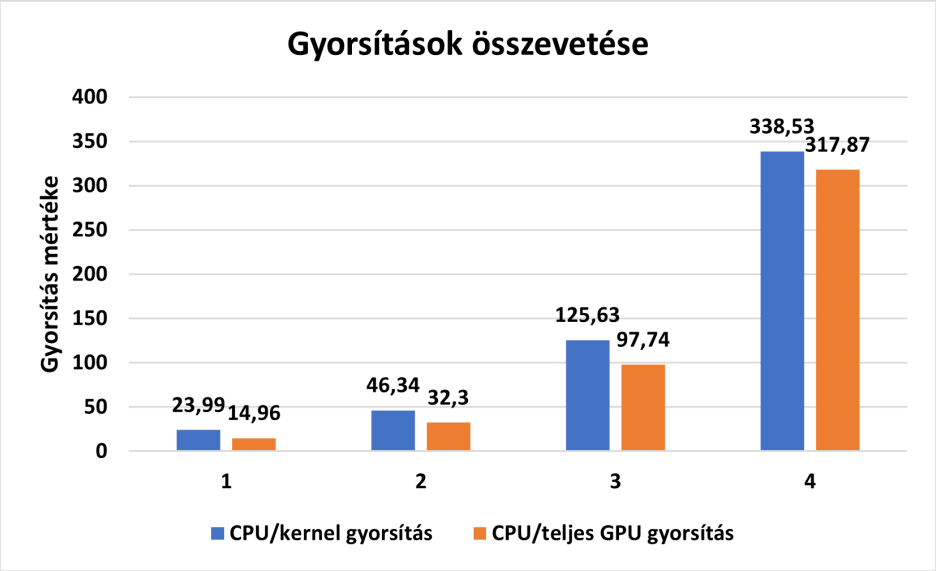
A grafikon azt mutatja, hogy mennyi ideig fut maga a GPU-kernel, vagyis az a rész, ahol a tényleges számítás történik, adatmásolás nélkül. A görbe jól láthatóan emelkedik, ahogy nő a mátrix mérete: minél nagyobb a mátrix, annál több számítást kell elvégezni, így a kernel futási ideje is hosszabb lesz. A növekedés azonban sokkal lassabb, mint a CPU esetében, mert a GPU több százezer vagy akár több millió szálra osztja szét a munkát. Kisebb mátrixoknál a kernel futási ideje szinte elhanyagolható, néhány ezredmásodperc alatt lefut. Ahogy viszont a mátrixméret nő, a GPU-nak is egyre több műveletet kell elvégeznie, ezért a futási idő fokozatosan emelkedik. Összességében a grafikon azt üzeni, hogy a GPU kernel rendkívül hatékonyan dolgozik, és még nagy mátrixok esetén is jóval gyorsabban végez, mint a CPU. A futási idő növekedése természetes, de a GPU párhuzamos felépítése miatt sokkal jobban skálázódik a feladat méretével.



5. ábra. Teljes GPU futási idő

6.5. 5.ábra magyarázata

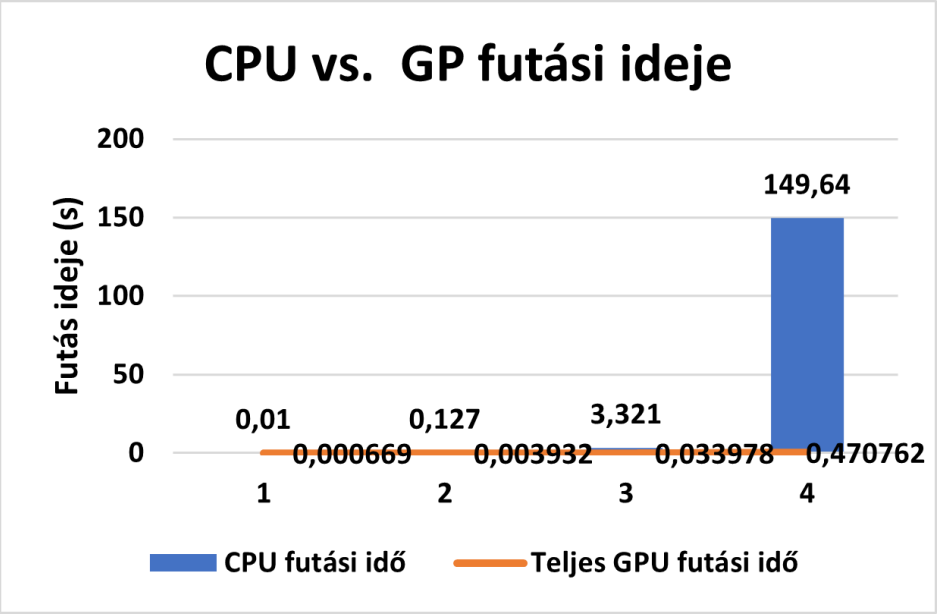
Ez a grafikon azt mutatja, hogy mennyi időbe telik a GPU-nak elvégezni a teljes mátrixszorzást adatmásolással együtt. A görbe jól láthatóan emelkedik, ahogy nő a mátrix mérete: minél nagyobb a mátrix, annál több adatot kell átmásolni a CPU és a GPU között, és annál több számítást kell elvégezni. A növekedés azonban még így is sokkal kedvezőbb, mint a CPU esetében, mert a GPU rengeteg szálon dolgozik egyszerre. Kisebb mátrixoknál a teljes futási idő szinte elhanyagolható, néhány ezredmásodperc alatt lezajlik minden. Ahogy viszont a mátrixméret nő, a másolási idő és a számítási idő együtt egyre nagyobb lesz, ezért a grafikon meredeken emelkedik. A lényeg mégis az, hogy a GPU még a legnagyobb mátrixoknál is nagyságrendekkel gyorsabb marad, mint a CPU — még akkor is, ha a másolási költség hozzáadódik. Összességében a grafikon azt üzeni, hogy a GPU teljes futási ideje természetesen nő a feladat méretével, de a párhuzamos működés miatt még így is sokkal hatékonyabb, mint a CPU.



6. ábra. Gyorsítások összevetése

6.6. 6.ábra magyarázata

A grafikon azt mutatja meg, hogy mennyivel gyorsabb a GPU a CPU-hoz képest különböző mátrixméreteknél, és külön választja a kernel gyorsítást (csak a számítás ideje) és a teljes GPU gyorsítást (számítás + adatmásolás). Jól látszik, hogy mindkét gyorsítási érték folyamatosan nő, ahogy a mátrix mérete nagyobb lesz. A kék oszlopok (CPU/kernel) mindig magasabbak, mert a kernel futási ideje önmagában nagyon gyors, és nem tartalmazza a másolási költséget. A narancssárga oszlopok (CPU/teljes GPU) kicsit alacsonyabbak, de így is hatalmas gyorsulást mutatnak. A grafikon lényege az, hogy a GPU előnye a méret növekedésével egyre látványosabb: kis mátrixoknál még „csak” néhányszoros gyorsulást látunk, de nagyobb méreteknél már több százszorosat. Ez azt bizonyítja, hogy a GPU akkor működik igazán hatékonyan, amikor sok párhuzamos feladatot kell egyszerre elvégezni. A különbség a két oszloptípus között pedig azt mutatja, hogy az adatmásolás lassít ugyan valamennyit, de a GPU előnyét nem tudja érdemben csökkenteni. Összességében a grafikon azt üzeni, hogy a GPU gyorsítása nemcsak stabilan nő, hanem nagy mátrixoknál már óriási mértéket ölt — és még a másolási költségek mellett is messze felülmúlja a CPU teljesítményét.



7. ábra. CPU vs. GPU Futási ideje

6.7. 7.ábra magyarázata

A grafikon nagyon látványosan mutatja meg, mekkora különbség van a CPU és a GPU között, amikor nagy mátrixokat kell összeszorozni. A kék oszlopok a CPU futási idejét mutatják, az narancs vonal pedig a teljes GPU futási időt (számítás + adatmásolás). Már az első pontnál is jól látszik, hogy a GPU sokkal gyorsabb, de ahogy nő a mátrix mérete, a különbség egyszerűen „elszáll”. Kisebb méreteknél a CPU még elfogadhatóan teljesít, de 512-es és 1024-es mátrixoknál már több nagyságrenddel lassabb lesz, míg a GPU továbbra is nagyon gyors marad. A legnagyobb méretnél (2048) a CPU futási ideje már közel 150 másodperc, miközben a GPU ugyanezt kevesebb mint fél másodperc alatt elvégzi. Ez óriási különbség, és tökéletesen megmutatja, hogy a CPU soros működése mennyire nem bírja a nagy számítási terhelést, míg a GPU több millió szálon párhuzamosan dolgozik. Összességében a grafikon azt üzeni, hogy a GPU nemcsak gyorsabb, hanem a feladat méretének növekedésével egyre jobban „elhúz” a CPU mellett. A CPU futási ideje meredeken nő, a GPU-é viszont sokkal lassabban, ezért a különbség nagy mátrixoknál már drámai. Ezért használják a modern, nagy számításigényű alkalmazásokban szinte mindenhol a GPU-t.

7. Összefoglalás

A mérések és az ábrák alapján egyértelműen megállapítható, hogy a GPU jelentős teljesítményelőnyt biztosít a CPU-val szemben mátrixszorzás esetén. Míg a CPU sorosan, kevés szálon végzi a számításokat, addig a GPU több millió párhuzamos szál segítségével osztja fel a feladatot, ami jelentős gyorsulást eredményez.

Az 1. ábra jól szemlélteti, hogy a CPU futási ideje a mátrix méretének növekedésével drasztikusan emelkedik, míg a 4. ábra azt mutatja, hogy a GPU kernel futási ideje ennél jóval lassabban növekszik a párhuzamos végrehajtás miatt.

A 2. ábra bemutatja, hogy a GPU által indított szálak száma a mátrix méretével négyzetesen nő, amely biztosítja a nagyfokú párhuzamosságot. A 3. ábra alapján az egy szálra jutó futási idő alacsony marad, ami azt jelzi, hogy a GPU hatékonyan osztja szét a részfeladatokat.

Az implementáció során a globális memória használata mellett lokális (local) memória is alkalmazásra került a kernelben. Ennek során a mátrix blokkok (tile-ok) a gyorsabb elérésű lokális memóriába kerültek betöltésre, majd ezekből történt a részszámítás. Ez jelentősen csökkentette a globális memóriaelérések számát, ami a memória-hozzáférés szűk keresztmetszetét mérsékelte, és ezáltal tovább javította a teljesítményt.

Az 5. ábra azt mutatja, hogy a teljes GPU futási idő (adatmásolással együtt) ugyan növekszik a mátrixmérettel, de továbbra is jelentősen alacsonyabb marad a CPU futási idejénél. Ezt a különbséget a 7. ábra is jól szemlélteti, ahol a CPU és GPU közötti teljesítménykülönbség nagy méreteknél drámai mértékű.

A gyorsítás mértékét a 6. ábra mutatja, ahol látható, hogy mind a kernel szintű, mind a teljes GPU gyorsítás növekszik a mátrix méretével. n=2048 esetén a GPU több mint 300-szoros gyorsulást ért el a CPU-hoz képest.

Összességében elmondható, hogy a GPU különösen nagy méretű, számításigényes feladatok esetén nyújt kiemelkedő teljesítményt. A párhuzamos feldolgozás és a lokális memória (tiling) használata együtt jelentősen csökkenti a memóriaelérések költségét, így tovább javítja a végrehajtási hatékonyságot. A GPU előnye ezért nemcsak a párhuzamosságban, hanem a memória-hierarchia hatékony kihasználásában is rejlik.

Irodalom

[1] G. H. Golub and C. F. Van Loan: *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.

[2] V. Strassen: Gaussian Elimination is Not Optimal. *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.
https://doi.org/10.1007/BF02165411

[3] J. Nickolls, I. Buck, M. Garland, and K. Skadron: Scalable Parallel Programming with CUDA. *Queue*, vol. 6, no. 2, pp. 40–53, 2008.
https://doi.org/10.1145/1365490.1365500

[4] NVIDIA Corporation: *CUDA C Programming Guide*, 2024.
https://docs.nvidia.com/cuda/cuda-c-programming-guide/